

Product Requirements Document (PRD)

Multi-Agent AI Collaboration System

1. What is a PRD?

A Product Requirements Document (PRD) defines what a product should do, why it is needed, and what features and requirements it should have. It provides a clear direction for the development team before implementation.

For this project, the PRD defines how multiple specialized AI agents collaborate to solve complex tasks and generate a final response.

2. Project Overview

The **Multi-Agent AI Collaboration System** is an Artificial Intelligence system where multiple specialized AI agents work together to complete complex tasks.

Instead of one agent handling everything, different agents perform specific roles such as task management, research, analysis, writing, and reviewing.

Basic Flow:

User Request → Manager → Specialized Agents → Reviewer → Final Response

3. Problem Statement

A single AI agent may need to research, analyze, write, and review information at the same time, making complex tasks difficult to manage effectively.

This project solves this problem by dividing tasks among specialized AI agents and coordinating their outputs to produce a better final response.

4. Target Users

- Students
- Researchers
- Developers
- Professionals
- Users handling complex tasks

5. Project Goals

- Enable multiple AI agents to collaborate on complex tasks.
- Automatically divide complex requests into smaller tasks.
- Assign tasks to suitable specialized agents.
- Improve the organization and quality of AI-generated responses.
- Review and improve the generated output.

6. Project Objectives

The system should:

1. Accept a complex user task.
2. Break it into smaller subtasks.
3. Assign subtasks to suitable AI agents.
4. Allow agents to share information and results.
5. Generate a structured response.
6. Review and improve the response.
7. Deliver the final response to the user.

7. Main AI Agents

Manager Agent

- Understands the user request.
- Decomposes the task.
- Assigns tasks and coordinates the workflow.

Research Agent

- Collects relevant information for its assigned task.

Analysis Agent

- Analyzes information and identifies important findings.

Writer Agent

- Converts research and analysis into a structured response or report.

Reviewer Agent

- Checks the response for missing or inconsistent information and provides improvement feedback.

8. Key Features and Requirements

- **User Input:** Accept complex tasks or questions.
- **Task Decomposition:** Break complex requests into smaller tasks.
- **Agent Collaboration:** Allow agents to exchange information and results.
- **Shared State:** Maintain information generated during the workflow.
- **Response Generation:** Generate a clear and structured final response.
- **Response Review:** Check and improve the generated response.
- **Activity Tracking:** Display or record agent activities.

- **Error Handling:** Handle incomplete or failed agent outputs appropriately.

9. Functional Requirements

The system must be able to:

- Accept user tasks.
- Decompose tasks and assign them to appropriate agents.
- Pass information between agents.
- Maintain task state.
- Generate, review, and improve responses.
- Display the final result.
- Handle agent failures or incomplete outputs.

10. Non-Functional Requirements

The system should be:

- **Reliable** – provide consistent results.
- **Efficient** – minimize unnecessary agent interactions.
- **Scalable** – support additional agents in the future.
- **User-friendly** – provide a simple interface.
- **Maintainable** – keep agents and workflows organized.
- **Secure** – protect user and system information.

11. Example Use Case

For a request such as:

“Analyze the impact of Artificial Intelligence on education and prepare a report.”

The system can work as:

Manager → Research Agent → Analysis Agent → Writer Agent → Reviewer Agent → Final Report

This demonstrates how specialized agents collaborate to solve a complex task.

12. Expected Outcomes

The project is expected to:

- Demonstrate collaboration between multiple AI agents.
- Automatically decompose complex tasks.
- Generate organized and structured responses.
- Improve outputs through automated review.

- Provide a practical multi-agent AI system.

13. Success Criteria

The project will be successful if it can:

- Understand user tasks.
- Correctly divide tasks into subtasks.
- Assign tasks to suitable agents.
- Enable effective agent communication.
- Generate meaningful final responses.
- Review and improve outputs.
- Complete tasks reliably.

14. Project Scope

Initial Scope

- Multiple specialized AI agents
- Task decomposition and coordination
- Shared workflow/state
- Response generation and review
- User interface and activity tracking

Future Scope

- Add more specialized agents.
- Integrate additional tools and knowledge sources.
- Improve agent memory and decision-making.
- Support domain-specific applications.

15. How a PRD Helps in the Project

Planning

Defines the goals, users, features, requirements, and expected outcomes before development.

Organizing

Breaks the project into clear components such as agents, workflows, collaboration, and review.

Development

Provides developers with a clear guide for implementing the required features.

Testing

Requirements and success criteria help verify whether the system works correctly.

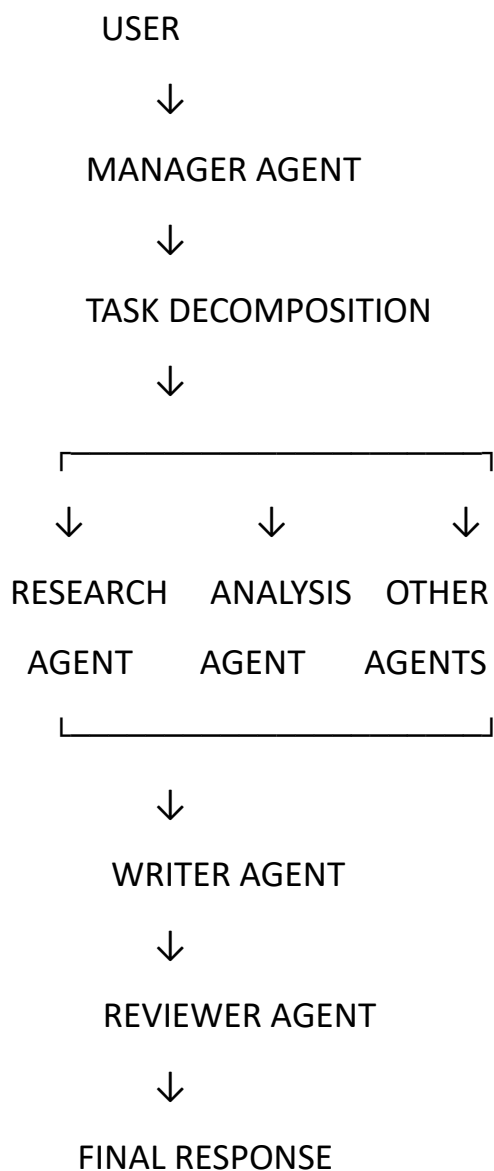
Team Communication

Helps team members understand the **project purpose, features, and responsibilities**.

Successful Project Building

Reduces confusion, avoids unnecessary features, and keeps development focused on the project goals.

16. Overall System Flow



Technical Requirements Document (TRD)

Multi-Agent AI Collaboration System

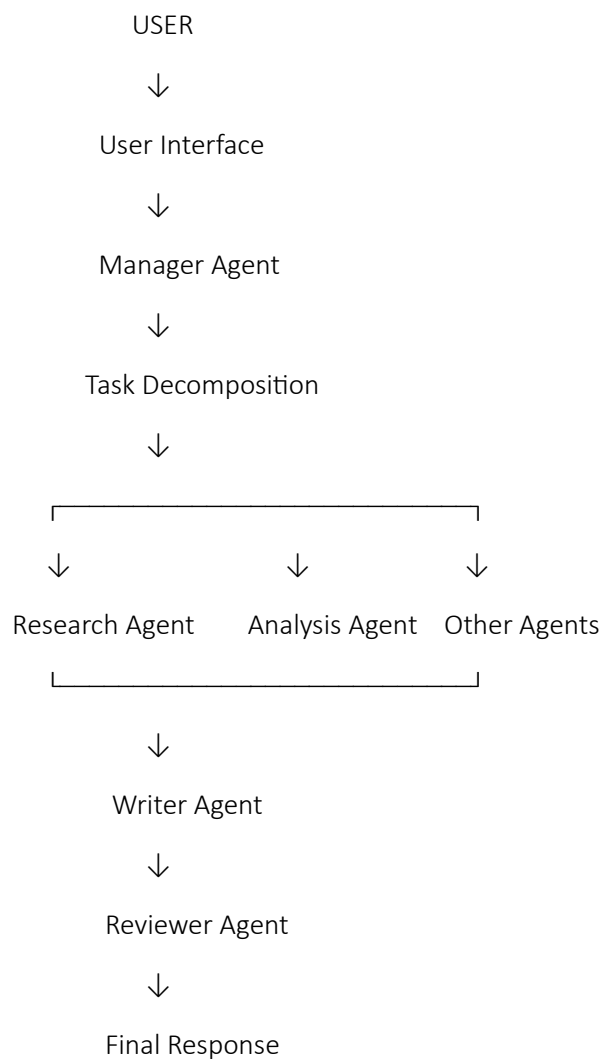
1. What is a TRD?

A Technical Requirements Document (TRD) describes how a project will be technically developed and implemented. It defines the system architecture, technologies, tools, workflows, and technical requirements needed to build the project.

For this project, the TRD explains how multiple AI agents communicate, collaborate, process tasks, and generate the final response.

2. Technical Architecture

The system follows a multi-agent architecture where a **Manager Agent** coordinates specialized AI agents.



Main Components

User Interface

Accepts the user's task and displays the final response and agent activity.

Manager Agent

Controls the workflow, breaks the task into subtasks, and assigns them to suitable agents.

Specialized Agents

Perform specific tasks such as research, analysis, and writing.

Shared State/Workflow

Stores and passes relevant information between agents during task execution.

Reviewer Agent

Checks the generated response and provides feedback for improvement.

3. Implementation Details

The project can be implemented using the following process:

Step 1 – Receive User Request

The user submits a complex question or task through the interface.

Step 2 – Task Decomposition

The Manager Agent analyzes the request and creates smaller subtasks.

Step 3 – Agent Assignment

Each subtask is assigned to the appropriate specialized AI agent.

Step 4 – Agent Execution

Agents perform their assigned tasks and generate results.

Step 5 – Information Sharing

The results are stored in a shared workflow/state and made available to other agents.

Step 6 – Response Generation

The Writer Agent combines the available results into a structured response.

Step 7 – Review

The Reviewer Agent checks the response and provides feedback if improvements are needed.

Step 8 – Final Output

The improved response is presented to the user.

4. Proposed Technologies

Programming Language

Python – Used for implementing the complete AI system and agent workflows.

Large Language Model (LLM)

An LLM API – Used as the intelligence layer for understanding tasks, reasoning, generating content, and communicating through agents.

Multi-Agent Framework

LangGraph – Used to design and manage the multi-agent workflow, agent communication, and shared state.

Backend

FastAPI – Used to create APIs and connect the AI system with the user interface.

Frontend

Streamlit – Used to create a simple and interactive user interface.

Database

SQLite – Used for lightweight storage of application data when required.

AI/Utility Libraries

Relevant Python libraries can be used for tasks such as data processing, API communication, and application development.

5. Tools Used

- **VS Code** – Code development and project management.
- **Git & GitHub** – Version control and source-code management.
- **Python Virtual Environment** – Managing project dependencies.
- **API Testing Tools** – Testing backend APIs when required.

6. Technical Workflow

User Input



Manager Agent



Task Decomposition



Task Assignment



Specialized Agents



Shared State



Writer Agent



Reviewer Agent



Final Response

The workflow allows each agent to focus on its specific responsibility while the Manager Agent coordinates the complete process.

7. Technical Requirements

The system requires:

- A Python development environment.
- Access to a suitable LLM API.
- Required Python libraries and dependencies.
- Multi-agent workflow framework.
- Internet connectivity when external APIs or online information sources are used.
- Sufficient computing resources for running the application and AI workflows.
- Secure management of API keys and user information.

8. Error Handling

The system should handle:

- Agent failures.
- Missing or incomplete responses.
- API errors.
- Invalid user requests.
- Timeouts or unavailable services.

If an agent fails, the system should provide an appropriate fallback or retry instead of stopping the entire workflow.

9. How a TRD Helps in Developing and Executing the Project

Technical Direction

Defines how the system should be designed and implemented.

Technology Selection

Clearly identifies the programming language, frameworks, tools, and other technologies required.

Development

Breaks implementation into clear stages such as **task decomposition, agent execution, collaboration, generation, and review.**

Testing and Debugging

Provides a technical reference for checking each component and identifying problems.

Team Collaboration

Helps developers understand the system architecture, workflows, and technical responsibilities.

Future Improvements

Makes it easier to add new agents, tools, workflows, and capabilities without redesigning the entire system.

10. Expected Technical Outcome

The final system should provide a working **multi-agent AI workflow** where:

User Request → Task Decomposition → Agent Collaboration → Response Generation → Review → Final Response

The architecture should be modular so that new agents and capabilities can be added in the future.

Project Abstract

Multi-Agent AI Collaboration System

The Multi-Agent AI Collaboration System is an Artificial Intelligence system designed to solve complex tasks by using multiple specialized AI agents that work together. Instead of relying on a single AI agent, the system assigns different responsibilities such as **task management**, **research**, **analysis**, **content generation**, and **review** to different agents.

Problem Being Addressed

A single AI agent may need to perform several different tasks simultaneously, such as understanding a problem, researching information, analyzing results, generating content, and checking the final response. Handling all these responsibilities together can make complex tasks difficult to organize and may affect the quality of the output.

Proposed Solution

The proposed system uses a **Manager Agent** to understand and decompose the user's request into smaller tasks. These tasks are assigned to specialized agents such as the **Research Agent**, **Analysis Agent**, **Writer Agent**, and **Reviewer Agent**. The agents share relevant information through a common workflow, and the Reviewer Agent checks the generated response before the final result is delivered to the user.

Expected Outcomes and Impact

The system is expected to:

- Automate the decomposition of complex tasks.
- Enable effective collaboration between multiple AI agents.
- Generate structured and meaningful responses.
- Improve outputs through automated review.
- Provide a flexible architecture that can support additional agents and capabilities.

How the Project Solves the Problem

The system solves the identified problem by **dividing complex work among specialized AI agents instead of making one agent handle every responsibility**. The Manager coordinates the agents, their results are combined through the workflow, and the Reviewer checks the final output. This creates a more organized and collaborative approach to solving complex AI tasks.

In simple terms:

One complex task → Multiple specialized AI agents → Collaboration → Review → Better-organized final response.

Visual Representation





TECHNICAL REQUIREMENTS DOCUMENT (TRD)

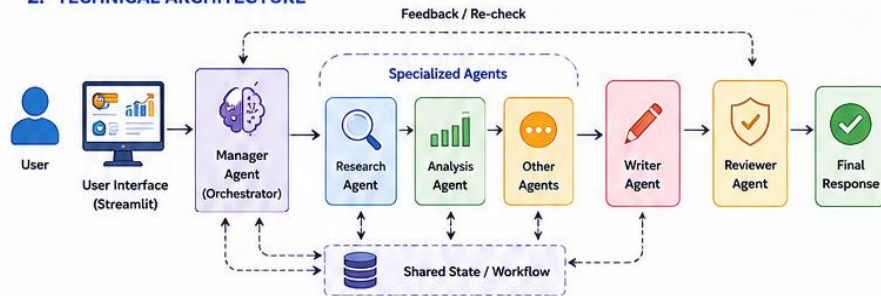
MULTI-AGENT AI COLLABORATION SYSTEM



1. WHAT IS A TRD?

A Technical Requirements Document (TRD) describes how the project will be developed and implemented. It defines the architecture, technologies, tools, workflows, and technical requirements needed to build the system.

2. TECHNICAL ARCHITECTURE



3. IMPLEMENTATION DETAILS

- 1. Receive User Request**
User enters a complex task through the interface.
- 2. Task Decomposition**
Manager Agent breaks the task into smaller subtasks.
- 3. Agent Assignment**
Subtasks are assigned to suitable specialized agents.
- 4. Agent Execution**
Agents perform their assigned tasks and generate results.
- 5. Information Sharing**
Results are stored in the shared state and available to others.
- 6. Response Generation**
Writer Agent combines the results into a structured response.
- 7. Review**
Reviewer Agent checks and provides feedback if needed.
- 8. Final Output**
Improved response is delivered to the user.



4. TECHNOLOGIES USED



Python
Main programming language used for building the entire system.



LLM (API)
Large Language Model API used for reasoning, generating responses and agent communication.



LangGraph
Framework used for building multi-agent workflows, state management and agent communication.



FastAPI
Used to build backend APIs and connect the system with the frontend.



Streamlit
Used to create a simple and interactive web interface.



SQLite
Lightweight database used for storing application data when required.



Python Libraries / Utilities
Used for data handling, API communication, environment management, and other supporting functionalities.

5. TOOLS USED



VS Code
Code editor and development



Git & GitHub
Version control and source code management



Postman
API testing and debugging



Python venv
Virtual environment and dependency management



Draw.io / Lucidchart
Designing system architecture and workflows

6. TECHNICAL WORKFLOW



7. TECHNICAL REQUIREMENTS

- Python 3.9+
- Access to LLM API (e.g., OpenAI API)
- Required Python libraries and dependencies
- Multi-agent workflow framework (LangGraph)
- Internet connection for API / information access
- Sufficient system resources (CPU, RAM, Storage)
- Secure API key and data management

8. ERROR HANDLING



- Handle agent failures and retries
- Handle API errors and timeouts
- Handle missing or incomplete outputs
- Handle invalid user requests
- Provide meaningful error messages and fallback responses

9. HOW TRD HELPS IN DEVELOPMENT AND EXECUTION



Technical Direction – Provides a clear plan for designing and implementing the system.



Technology Selection – Identifies the right technologies and tools for the project.



Development – Breaks the system into components and implementation stages.



Testing & Debugging – Helps in testing each module and fixing technical issues.



Team Collaboration – Helps the team understand architecture, workflows and responsibilities.



Future Improvements – Makes it easy to add new agents, tools and features.

10. EXPECTED TECHNICAL OUTCOME



★ The system will provide a modular, scalable and efficient multi-agent architecture that can handle complex tasks and can be extended for future applications.